

SKALA FULL-STACK ENGINEERING · 종합실습 4

E-Commerce 매출 분석 및 정리

작성자 : 판교 4반 · 최승우

작성일 : 2026-08-14

Executive Summary

분석의 핵심은 ‘누가 요청했는가’보다 ‘무엇을 한 건으로 볼 것인가’를 먼저 정하는 것이었다. 주문과 주문상품의 1:N 관계를 그대로 집계하면 주문 수와 평균 주문 금액이 왜곡되므로, Q2·Q5·Q9는 주문 단위 선집계를 기준으로 재작성했다. 상관 서브쿼리가 반복되던 Q6·Q7·Q10은 윈도우 함수·JOIN·조건부 집계로 단순화했다.

분석 DB	skala_ecom_practice4 / user=chltmddn5843
데이터 규모	고객 3,000 · 상품 600 · 주문 9,890 · 주문상품 26,716
유효 매출 상태	paid · shipped · delivered
공통 매출식	$\text{order_items.line_total} = \text{unit_price} \times \text{qty} - \text{discount}$
측정 방식	비교표는 EXPLAIN 3회 중앙값 · 원문 로그 화면은 실행 당시 1회 실제 출력
권한 주의 본 실습은 로컬 관리자 계정으로 수행했다. 운영 환경에서는 분석용 읽기 전용 계정과 Materialized View 유지보수 계정을 분리하고 최소 권한만 부여해야 한다.	

요청 대응 원칙

- 기간·상태·집계 단위·출력 건수를 SQL보다 먼저 확정한다.
- 랜덤 데이터에서는 수치보다 결과가 바뀌어도 유지되는 판정 로직을 검증한다.
- 실행시간만 보지 않고 실제 행 수와 Buffers를 함께 비교한다.
- 결과에는 기준시각과 가정을 붙이고, 관찰 비교를 인과효과로 표현하지 않는다.

Q1. 최근 한 달 실제 총매출

요청 부서	재무
요청 목적	최근 한 달 실제 매출 규모 확인
판정 기준	롤링 1개월·유효 주문 상태

튜닝 전

직접 조인 집계

```
SELECT
  sum(oi.line_total) AS total_revenue
FROM ecom.orders o
JOIN ecom.order_items oi
  ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
AND o.order_ts >= now() - interval '1 month'
```

튜닝 전 실행 결과

Q01 튜닝 전 결과

```
total_revenue
-----
2,063,896.36
rows=1 | median execution=5.370 ms
```

튜닝 전 성능 원문 로그 · Q01_plan_before.png

```
Q01 최근 한 달 실제 총매출 [before]

PERFORMANCE SQL
EXPLAIN (ANALYZE, BUFFERS)
SELECT
  sum(oi.line_total) AS total_revenue
FROM ecom.orders o
JOIN ecom.order_items oi
  ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
AND o.order_ts >= now() - interval '1 month'

PERFORMANCE RESULT
Aggregate (cost=947.69..947.70 rows=1 width=32) (actual time=8.418..8.422 rows=1 loops=1)
 Buffers: shared hit=360
-> Hash Join (cost=338.26..933.59 rows=5640 width=6) (actual time=3.514..7.968 rows=5756 loops=1)
   Hash Cond: (oi.order_id = o.order_id)
   Buffers: shared hit=360
   -> Seq Scan on order_items oi (cost=0.00..525.16 rows=26716 width=14) (actual time=0.005..1.752
rows=26716 loops=1)
     Buffers: shared hit=258
   -> Hash (cost=312.16..312.16 rows=2088 width=8) (actual time=3.489..3.490 rows=2111 loops=1)
     Buckets: 4096 Batches: 1 Memory Usage: 115kB
     Buffers: shared hit=102
     -> Seq Scan on orders o (cost=0.00..312.16 rows=2088 width=8) (actual time=0.010..3.245
rows=2111 loops=1)
       Filter: ((order_status = ANY ('paid,shipped,delivered')::text[])) AND (order_ts >=
(now() - '1 mon'::interval)))
       Rows Removed by Filter: 7779
       Buffers: shared hit=102

Planning:
 Buffers: shared hit=357
Planning Time: 1.299 ms
Execution Time: 8.473 ms
```

튜닝 포인트

변경 기간·상태 대상 주문을 먼저 명명해 조건과 집계 경로를 분리

튜닝 후

```
WITH recent_orders AS (
  SELECT
    order_id
  FROM ecom.orders
  WHERE order_status IN ('paid', 'shipped', 'delivered')
  AND order_ts >= now() - interval '1 month'
)

SELECT
  sum(oi.line_total) AS total_revenue
FROM recent_orders ro
JOIN ecom.order_items oi
  ON oi.order_id = ro.order_id
```

튜닝 후 실행 결과

Q01 튜닝 후 결과

```
total_revenue
-----
2,063,896.36
rows=1 | median execution=4.214 ms
```

튜닝 후 성능 원문 로그 · Q01_plan_after.png

```
Q01 최근 한 달 실제 총매출 [after]

PERFORMANCE SQL
EXPLAIN (ANALYZE, BUFFERS)
WITH recent_orders AS (
  SELECT
    order_id
  FROM ecom.orders
  WHERE order_status IN ('paid', 'shipped', 'delivered')
  AND order_ts >= now() - interval '1 month'
)

SELECT
  sum(oi.line_total) AS total_revenue
FROM recent_orders ro
JOIN ecom.order_items oi
  ON oi.order_id = ro.order_id

PERFORMANCE RESULT
Aggregate (cost=947.69..947.70 rows=1 width=32) (actual time=8.299..8.303 rows=1 loops=1)
 Buffers: shared hit=360
-> Hash Join (cost=338.26..933.59 rows=5640 width=6) (actual time=3.102..7.814 rows=5756 loops=1)
   Hash Cond: (oi.order_id = orders.order_id)
   Buffers: shared hit=360
   -> Seq Scan on order_items oi (cost=0.00..525.16 rows=26716 width=14) (actual time=0.004..1.892
rows=26716 loops=1)
     Buffers: shared hit=258
   -> Hash (cost=312.16..312.16 rows=2088 width=8) (actual time=3.078..3.078 rows=2111 loops=1)
     Buckets: 4096 Batches: 1 Memory Usage: 115kB
     Buffers: shared hit=102
     -> Seq Scan on orders (cost=0.00..312.16 rows=2088 width=8) (actual time=0.009..2.857
rows=2111 loops=1)
       Filter: ((order_status = ANY ('{paid,shipped,delivered} '::text[])) AND (order_ts >=
(now() - '1 mon'::interval)))
       Rows Removed by Filter: 7779
       Buffers: shared hit=102

Planning:
 Buffers: shared hit=357
Planning Time: 1.156 ms
Execution Time: 8.346 ms
```

튜닝 전후 실행계획 비교

측정 항목	튜닝 전	튜닝 후
Execution Time	5.370 ms	4.214 ms
Planning Time	0.099 ms	0.081 ms
Actual Rows	1	1
Shared Hit	360	360
Shared Read	0	0
Top Plan Node	Aggregate	Aggregate
결과 동일	-	YES

업무 판단 및 요청 부서 전달안

튜닝 판단

PostgreSQL이 CTE를 인라인하면 계획은 같을 수 있다. 성능보다 요청 조건의 가독성이 핵심이다.

재무 전달안

최근 한 달 실제 매출은 2,063,896.36이다.

Q2. 월별 주문 수·매출·AOV

요청 부서	재무·경영
요청 목적	월별 주문 수·매출·AOV 추이
판정 기준	달력 월·주문 단위 평균

튜닝 전

주문상품 행에서 COUNT DISTINCT와 합계를 동시에 수행

```
SELECT
  date_trunc('month', o.order_ts)::date AS month,
  count(DISTINCT o.order_id) AS order_count,
  sum(oi.line_total) AS revenue,
  sum(oi.line_total) / count(DISTINCT o.order_id) AS aov
FROM ecom.orders o
JOIN ecom.order_items oi
  ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
GROUP BY date_trunc('month', o.order_ts)::date
ORDER BY month
```

튜닝 전 실행 결과

Q02 튜닝 전 결과

month	order_count	revenue	aov
2026-04-01	823	728,085.26	884.67
2026-05-01	1873	1,666,271.77	889.63
2026-06-01	1915	1,687,798.91	881.36
2026-07-01	2144	2,034,521.03	948.94
2026-08-01	858	871,025.70	1,015.18

rows=5 | median execution=10.463 ms

튜닝 전 성능 원문 로그 · Q02_plan_before.png

```

Q02 월별 주문 수-매출-AOV [before]

PERFORMANCE SQL
EXPLAIN (ANALYZE, BUFFERS)
SELECT
    date_trunc('month', o.order_ts)::date AS month,
    count(DISTINCT o.order_id) AS order_count,
    sum(oi.line_total) AS revenue,
    sum(oi.line_total) / count(DISTINCT o.order_id) AS aov
FROM ecom.orders o
JOIN ecom.order_items oi
    ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
GROUP BY date_trunc('month', o.order_ts)::date
ORDER BY month

PERFORMANCE RESULT
GroupAggregate (cost=2504.56..2881.51 rows=7613 width=76) (actual time=14.644..16.750 rows=5 loops=1)
  Group Key: ((date_trunc('month'::text, o.order_ts)::date)
  Buffers: shared hit=366
  -> Sort (cost=2504.56..2555.98 rows=20565 width=18) (actual time=14.462..15.102 rows=20557 loops=1)
    Sort Key: ((date_trunc('month'::text, o.order_ts)::date), o.order_id
    Sort Method: quicksort  Memory: 1572kB
    Buffers: shared hit=366
    -> Hash Join (cost=333.15..1031.30 rows=20565 width=18) (actual time=1.588..10.974 rows=20557
    loops=1)
      Hash Cond: (oi.order_id = o.order_id)
      Buffers: shared hit=360
      -> Seq Scan on order_items oi (cost=0.00..525.16 rows=26716 width=14) (actual
      time=0.003..1.413 rows=26716 loops=1)
        Buffers: shared hit=258
      -> Hash (cost=237.99..237.99 rows=7613 width=16) (actual time=1.566..1.566 rows=7613
      loops=1)
        Buckets: 8192  Batches: 1  Memory Usage: 421kB
        Buffers: shared hit=102
        -> Seq Scan on orders o (cost=0.00..237.99 rows=7613 width=16) (actual
        time=0.004..1.116 rows=7613 loops=1)
          Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
          Rows Removed by Filter: 2277
          Buffers: shared hit=102
    Planning:
      Buffers: shared hit=280
    Planning Time: 0.515 ms
    Execution Time: 16.797 ms

```

튜닝 포인트

변경 주문별 금액 선집계 후 월 집계

튜닝 후

```

WITH order_totals AS (
    SELECT
        o.order_id,
        date_trunc('month', o.order_ts)::date AS month,
        sum(oi.line_total) AS amount
    FROM ecom.orders o
    JOIN ecom.order_items oi
        ON oi.order_id = o.order_id
    WHERE o.order_status IN ('paid', 'shipped', 'delivered')
    GROUP BY
        o.order_id,
        date_trunc('month', o.order_ts)::date
)

SELECT
    month,
    count(*) AS order_count,
    sum(amount) AS revenue,
    ecom.safe_div(sum(amount), count(*)) AS aov
FROM order_totals
GROUP BY month
ORDER BY month

```

튜닝 후 실행 결과

Q02 튜닝 후 결과

month	order_count	revenue	aov
2026-04-01	823	728,085.26	884.67
2026-05-01	1873	1,666,271.77	889.63
2026-06-01	1915	1,687,798.91	881.36
2026-07-01	2144	2,034,521.03	948.94
2026-08-01	858	871,025.70	1,015.18
rows=5 median execution=7.926 ms			

튜닝 후 성능 원문 로그 · Q02_plan_after.png (1/2)

Q02 월별 주문 수·매출·AOV [after]

PERFORMANCE SQL
EXPLAIN (ANALYZE, BUFFERS)

```
WITH order_totals AS (
  SELECT
    o.order_id,
    date_trunc('month', o.order_ts)::date AS month,
    sum(oi.line_total) AS amount
  FROM ecom.orders o
  JOIN ecom.order_items oi
    ON oi.order_id = o.order_id
  WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  GROUP BY
    o.order_id,
    date_trunc('month', o.order_ts)::date
)
```

```
SELECT
  month,
  count(*) AS order_count,
  sum(amount) AS revenue,
  ecom.safe_div(sum(amount), count(*)) AS aov
FROM order_totals
GROUP BY month
ORDER BY month
```

PERFORMANCE RESULT

```
Sort (cost=1464.63..1465.13 rows=200 width=76) (actual time=13.735..13.737 rows=5 loops=1)
  Sort Key: ((date_trunc('month'::text, o.order_ts))::date)
  Sort Method: quicksort Memory: 25kB
  Buffers: shared hit=363
  -> HashAggregate (cost=1451.99..1456.99 rows=200 width=76) (actual time=13.726..13.728 rows=5
  loops=1)
    Group Key: ((date_trunc('month'::text, o.order_ts))::date)
    Batches: 1 Memory Usage: 40kB
    Buffers: shared hit=360
    -> HashAggregate (cost=1185.54..1318.76 rows=7613 width=44) (actual time=12.010..13.063
    rows=7613 loops=1)
      Group Key: o.order_id, (date_trunc('month'::text, o.order_ts))::date
      Batches: 1 Memory Usage: 3217kB
      Buffers: shared hit=360
      -> Hash Join (cost=333.15..1031.30 rows=20565 width=18) (actual time=1.071..9.196
      rows=20557 loops=1)
        Hash Cond: (oi.order_id = o.order_id)
        Buffers: shared hit=360
        -> Seq Scan on order_items oi (cost=0.00..525.16 rows=26716 width=14) (actual
        time=0.002..1.188 rows=26716 loops=1)
          Buffers: shared hit=258
        -> Hash (cost=237.99..237.99 rows=7613 width=16) (actual time=1.057..1.057
        rows=7613 loops=1)
          Buckets: 8192 Batches: 1 Memory Usage: 421kB
          Buffers: shared hit=102
          -> Seq Scan on orders o (cost=0.00..237.99 rows=7613 width=16) (actual
          time=0.002..0.732 rows=7613 loops=1)
            Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
            Rows Removed by Filter: 2277
            Buffers: shared hit=102

Planning:
  Buffers: shared hit=498
Planning Time: 0.507 ms
Execution Time: 13.810 ms
```


튜닝 후 성능 원문 로그 · Q02_plan_after.png (2/2)



튜닝 전후 실행계획 비교

측정 항목	튜닝 전	튜닝 후
Execution Time	10.463 ms	7.926 ms
Planning Time	0.062 ms	0.066 ms
Actual Rows	5	5
Shared Hit	360	360
Shared Read	0	0
Top Plan Node	Aggregate	Sort
결과 동일	-	YES

업무 판단 및 요청 부서 전달안

튜닝 판단

1:N 조인으로 부분 행을 주문 단위로 먼저 축소했다.

재무·경영 전달안

최신 월 (2026-08-01) 주문은 858건, 매출 871,025.70, AOV 1,015.18로 집계됐다. 부분 월 여부를 함께 전달해야 한다.

Q3. 최근 90일 카테고리 매출 Top 10

요청 부서	구매·MD
요청 목적	최근 90일 카테고리 성과 비교
판정 기준	카테고리 매출 Top 10

튜닝 전

전체 조인 후 기간 필터와 집계

```
SELECT
  c.category_id,
  c.category_name,
  sum(oi.line_total) AS revenue
FROM ecom.order_items oi
JOIN ecom.orders o
  ON o.order_id = oi.order_id
JOIN ecom.products p
  ON p.product_id = oi.product_id
JOIN ecom.categories c
  ON c.category_id = p.category_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  AND o.order_ts >= now() - interval '90 days'
GROUP BY
  c.category_id,
  c.category_name
ORDER BY
  revenue DESC,
  c.category_id
LIMIT 10
```

튜닝 전 실행 결과

Q03 튜닝 전 결과

```
category_id | category_name | revenue
-----+-----+-----
6           | Appliances    | 1,901,671.14
4           | Audio         | 1,256,556.19
13          | Outdoor       | 309,921.88
3           | Laptops       | 306,953.82
14          | Fitness       | 303,426.89
11          | Shoes         | 286,309.97
7           | Cookware      | 277,215.27
10          | Women         | 270,003.37
... 2 rows omitted
rows=10 | median execution=5.910 ms
```

튜닝 전 성능 원문 로그 · Q03_plan_before.png (1/2)

```

Q03 최근 90일 카테고리 매출 Top 10 [before]

PERFORMANCE SQL
EXPLAIN (ANALYZE, BUFFERS)
SELECT
    c.category_id,
    c.category_name,
    sum(oi.line_total) AS revenue
FROM ecom.order_items oi
JOIN ecom.orders o
    ON o.order_id = oi.order_id
JOIN ecom.products p
    ON p.product_id = oi.product_id
JOIN ecom.categories c
    ON c.category_id = p.category_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
AND o.order_ts >= now() - interval '90 days'
GROUP BY
    c.category_id,
    c.category_name
ORDER BY
    revenue DESC,
    c.category_id
LIMIT 10

PERFORMANCE RESULT
Limit (cost=1233.23..1233.25 rows=10 width=72) (actual time=11.279..11.283 rows=10 loops=1)
  Buffers: shared hit=374
  -> Sort (cost=1233.23..1235.90 rows=1070 width=72) (actual time=11.278..11.280 rows=10 loops=1)
    Sort Key: (sum(oi.line_total)) DESC, c.category_id
    Sort Method: quicksort Memory: 25kB
    Buffers: shared hit=374
    -> HashAggregate (cost=1196.73..1210.11 rows=1070 width=72) (actual time=11.249..11.259 rows=10
loops=1)
      Group Key: c.category_id
      Batches: 1 Memory Usage: 73kB
      Buffers: shared hit=368
      -> Hash Join (cost=439.59..1118.02 rows=15743 width=46) (actual time=3.496..9.888
rows=15883 loops=1)
        Hash Cond: (p.category_id = c.category_id)
        Buffers: shared hit=368
        -> Hash Join (cost=405.51..1042.45 rows=15743 width=14) (actual time=3.481..8.406
rows=15883 loops=1)
          Hash Cond: (oi.product_id = p.product_id)
          Buffers: shared hit=367
          -> Hash Join (cost=385.01..980.34 rows=15743 width=14) (actual
time=3.379..6.821 rows=15883 loops=1)
            Hash Cond: (oi.order_id = o.order_id)
            Buffers: shared hit=360
            -> Seq Scan on order_items oi (cost=0.00..525.16 rows=26716 width=22)
(actual time=0.002..1.117 rows=26716 loops=1)
              Buffers: shared hit=258
              -> Hash (cost=312.16..312.16 rows=5828 width=8) (actual
time=3.367..3.368 rows=5873 loops=1)
                Buckets: 8192 Batches: 1 Memory Usage: 294kB
                Buffers: shared hit=102
                -> Seq Scan on orders o (cost=0.00..312.16 rows=5828 width=8)
(actual time=0.005..2.962 rows=5873 loops=1)
                  Filter: ((order_status = ANY
('paid,shipped,delivered')::text[])) AND (order_ts >= (now() - '90 days'::interval)))
                  Rows Removed by Filter: 4017
                  Buffers: shared hit=102
                  -> Hash (cost=13.00..13.00 rows=600 width=16) (actual time=0.097..0.097
rows=600 loops=1)
                    Buckets: 1024 Batches: 1 Memory Usage: 37kB
                    Buffers: shared hit=7

```

튜닝 전 성능 원문 로그 · Q03_plan_before.png (2/2)

```

          Buffers: shared hit=7
          -> Seq Scan on products p (cost=0.00..13.00 rows=600 width=16) (actual
time=0.003..0.053 rows=600 loops=1)
            Buffers: shared hit=7
            -> Hash (cost=20.70..20.70 rows=1070 width=40) (actual time=0.011..0.011 rows=14
loops=1)
              Buckets: 2048 Batches: 1 Memory Usage: 17kB
              Buffers: shared hit=1
              -> Seq Scan on categories c (cost=0.00..20.70 rows=1070 width=40) (actual
time=0.005..0.007 rows=14 loops=1)
                Buffers: shared hit=1

Planning:
  Buffers: shared hit=354
  Planning Time: 0.645 ms
  Execution Time: 11.356 ms

```

튜닝 포인트

변경 최근 판매행을 먼저 좁힌 뒤 상품·카테고리 연결

튜닝 후

```
WITH recent_sales AS (
  SELECT
    oi.product_id,
    oi.line_total
  FROM ecom.orders o
  JOIN ecom.order_items oi
    ON oi.order_id = o.order_id
  WHERE o.order_status IN ('paid', 'shipped', 'delivered')
    AND o.order_ts >= now() - interval '90 days'
)

SELECT
  c.category_id,
  c.category_name,
  sum(rs.line_total) AS revenue
FROM recent_sales rs
JOIN ecom.products p
  ON p.product_id = rs.product_id
JOIN ecom.categories c
  ON c.category_id = p.category_id
GROUP BY
  c.category_id,
  c.category_name
ORDER BY
  revenue DESC,
  c.category_id
LIMIT 10
```

튜닝 후 실행 결과

Q03 튜닝 후 결과

category_id	category_name	revenue
6	Appliances	1,901,671.14
4	Audio	1,256,556.19
13	Outdoor	309,921.88
3	Laptops	306,953.82
14	Fitness	303,426.89
11	Shoes	286,309.97
7	Cookware	277,215.27
10	Women	270,003.37
... 2 rows omitted		

rows=10 | median execution=5.881 ms

튜닝 후 성능 원문 로그 · Q03_plan_after.png (1/2)

```

Q03 최근 90일 카테고리 매출 Top 10 [after]

PERFORMANCE SQL
EXPLAIN (ANALYZE, BUFFERS)
WITH recent_sales AS (
    SELECT
        oi.product_id,
        oi.line_total
    FROM ecom.orders o
    JOIN ecom.order_items oi
        ON oi.order_id = o.order_id
    WHERE o.order_status IN ('paid', 'shipped', 'delivered')
        AND o.order_ts >= now() - interval '90 days'
)

SELECT
    c.category_id,
    c.category_name,
    sum(rs.line_total) AS revenue
FROM recent_sales rs
JOIN ecom.products p
    ON p.product_id = rs.product_id
JOIN ecom.categories c
    ON c.category_id = p.category_id
GROUP BY
    c.category_id,
    c.category_name
ORDER BY
    revenue DESC,
    c.category_id
LIMIT 10

PERFORMANCE RESULT
Limit (cost=1233.23..1233.25 rows=10 width=72) (actual time=12.705..12.710 rows=10 loops=1)
  Buffers: shared hit=374
  -> Sort (cost=1233.23..1235.90 rows=1070 width=72) (actual time=12.704..12.706 rows=10 loops=1)
    Sort Key: (sum(oi.line_total)) DESC, c.category_id
    Sort Method: quicksort  Memory: 25kB
    Buffers: shared hit=374
    -> HashAggregate (cost=1196.73..1210.11 rows=1070 width=72) (actual time=12.671..12.683 rows=10
loops=1)
      Group Key: c.category_id
      Batches: 1  Memory Usage: 73kB
      Buffers: shared hit=368
      -> Hash Join (cost=439.59..1118.02 rows=15743 width=46) (actual time=3.649..11.126
rows=15883 loops=1)
        Hash Cond: (p.category_id = c.category_id)
        Buffers: shared hit=368
        -> Hash Join (cost=405.51..1042.45 rows=15743 width=14) (actual time=3.632..9.422
rows=15883 loops=1)
          Hash Cond: (oi.product_id = p.product_id)
          Buffers: shared hit=367
          -> Hash Join (cost=385.01..980.34 rows=15743 width=14) (actual
time=3.513..7.633 rows=15883 loops=1)
            Hash Cond: (oi.order_id = o.order_id)
            Buffers: shared hit=360
            -> Seq Scan on order_items oi (cost=0.00..525.16 rows=26716 width=22)
(actual time=0.003..1.382 rows=26716 loops=1)
              Buffers: shared hit=258
              -> Hash (cost=312.16..312.16 rows=5828 width=8) (actual
time=3.494..3.495 rows=5873 loops=1)
                Buckets: 8192  Batches: 1  Memory Usage: 294kB
                Buffers: shared hit=102
                -> Seq Scan on orders o (cost=0.00..312.16 rows=5828 width=8)
(actual time=0.005..2.987 rows=5873 loops=1)
                  Filter: (order_status = ANY(

```

튜닝 후 성능 원문 로그 · Q03_plan_after.png (2/2)

```

Filter: ((order_status = 'ANY'
('paid,shipped,delivered'))::text[])) AND (order_ts >= (now() - '90 days'::interval)))
Rows Removed by Filter: 4017
Buffers: shared hit=102
-> Hash (cost=13.00..13.00 rows=600 width=16) (actual time=0.112..0.112
rows=600 loops=1)
  Buckets: 1024 Batches: 1 Memory Usage: 37kB
  Buffers: shared hit=7
  -> Seq Scan on products p (cost=0.00..13.00 rows=600 width=16) (actual
time=0.003..0.063 rows=600 loops=1)
    Buffers: shared hit=7
  -> Hash (cost=20.70..20.70 rows=1070 width=40) (actual time=0.012..0.012 rows=14
loops=1)
    Buckets: 2048 Batches: 1 Memory Usage: 17kB
    Buffers: shared hit=1
    -> Seq Scan on categories c (cost=0.00..20.70 rows=1070 width=40) (actual
time=0.006..0.007 rows=14 loops=1)
      Buffers: shared hit=1
Planning:
  Buffers: shared hit=354
Planning Time: 1.326 ms
Execution Time: 12.777 ms

```

튜닝 전후 실행계획 비교

측정 항목	튜닝 전	튜닝 후
Execution Time	5.910 ms	5.881 ms
Planning Time	0.151 ms	0.145 ms
Actual Rows	10	10
Shared Hit	368	368
Shared Read	0	0
Top Plan Node	Limit	Limit
결과 동일	-	YES

업무 판단 및 요청 부서 전달안

튜닝 판단

소규모 데이터에서는 계획이 같을 수 있으나 분석 경계가 선명해졌다.

구매·MD 전달안

최근 90일 1위 카테고리는 Appliances, 매출은 1,901,671.14다. 순위는 랜덤 시드 재적재 시 바뀔 수 있다.

Q4. 상품별 누적매출 RANK Top 20

요청 부서	MD
요청 목적	누적매출 상위 상품 선별
판정 기준	RANK 기준 정확히 20개

튜닝 전

상품명까지 포함해 집계와 순위를 한 단계에서 처리

```
SELECT
  p.product_id,
  p.product_name,
  sum(oi.line_total) AS revenue,
  rank() OVER (
    ORDER BY sum(oi.line_total) DESC
  ) AS revenue_rank
FROM ecom.products p
JOIN ecom.order_items oi
  ON oi.product_id = p.product_id
JOIN ecom.orders o
  ON o.order_id = oi.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
GROUP BY
  p.product_id,
  p.product_name
ORDER BY
  revenue_rank,
  p.product_id
LIMIT 20
```

튜닝 전 실행 결과

Q04 튜닝 전 결과

```
product_id | product_name | revenue      | revenue_rank
-----|-----|-----|-----
457      | Product 208  | 2,340,762.83 | 1
588      | Product 510  | 998,376.91   | 2
85       | Product 112  | 21,508.92    | 3
317      | Product 156  | 17,799.75    | 4
347      | Product 466  | 17,102.35    | 5
467      | Product 58   | 16,838.52    | 6
234      | Product 174  | 16,782.15    | 7
540      | Product 359  | 15,995.89    | 8
... 12 rows omitted
rows=20 | median execution=5.500 ms
```

튜닝 전 성능 원문 로그 · Q04_plan_before.png (1/2)

```

Q04 상품별 누적매출 RANK Top 20 [before]

PERFORMANCE SQL
EXPLAIN (ANALYZE, BUFFERS)
SELECT
    p.product_id,
    p.product_name,
    sum(oi.line_total) AS revenue,
    rank() OVER (
        ORDER BY sum(oi.line_total) DESC
    ) AS revenue_rank
FROM ecom.products p
JOIN ecom.order_items oi
    ON oi.product_id = p.product_id
JOIN ecom.orders o
    ON o.order_id = oi.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
GROUP BY
    p.product_id,
    p.product_name
ORDER BY
    revenue_rank,
    p.product_id
LIMIT 20

PERFORMANCE RESULT
Limit (cost=1167.81..1167.86 rows=20 width=59) (actual time=15.491..15.495 rows=20 loops=1)
  Buffers: shared hit=373
  -> Sort (cost=1167.81..1169.31 rows=600 width=59) (actual time=15.489..15.490 rows=20 loops=1)
    Sort Key: (rank() OVER (?), p.product_id
    Sort Method: top-N heapsort  Memory: 26kB
    Buffers: shared hit=373
    -> WindowAgg (cost=1141.36..1151.85 rows=600 width=59) (actual time=15.306..15.406 rows=600
loops=1)
      Buffers: shared hit=370
      -> Sort (cost=1141.35..1142.85 rows=600 width=51) (actual time=15.299..15.312 rows=600
loops=1)
        Sort Key: (sum(oi.line_total)) DESC
        Sort Method: quicksort  Memory: 53kB
        Buffers: shared hit=370
        -> HashAggregate (cost=1106.16..1113.66 rows=600 width=51) (actual
time=15.162..15.209 rows=600 loops=1)
          Group Key: p.product_id
          Batches: 1  Memory Usage: 297kB
          Buffers: shared hit=367
          -> Hash Join (cost=353.65..1003.33 rows=20565 width=25) (actual
time=2.766..11.424 rows=20557 loops=1)
            Hash Cond: (oi.product_id = p.product_id)
            Buffers: shared hit=367
            -> Hash Join (cost=333.15..928.47 rows=20565 width=14) (actual
time=2.538..8.490 rows=20557 loops=1)
              Hash Cond: (oi.order_id = o.order_id)
              Buffers: shared hit=360
              -> Seq Scan on order_items oi (cost=0.00..525.16 rows=26716
width=22) (actual time=0.016..1.722 rows=26716 loops=1)
                Buffers: shared hit=258
                -> Hash (cost=237.99..237.99 rows=7613 width=8) (actual
time=2.493..2.493 rows=7613 loops=1)
                  Buckets: 8192  Batches: 1  Memory Usage: 362kB
                  Buffers: shared hit=102
                  -> Seq Scan on orders o (cost=0.00..237.99 rows=7613
width=8) (actual time=0.008..1.746 rows=7613 loops=1)
                    Filter: (order_status = ANY
('{paid,shipped,delivered'}::text[]))
                    Rows Removed by Filter: 2277
                    Buffers: shared hit=102

```

튜닝 전 성능 원문 로그 · Q04_plan_before.png (2/2)

```

      Buffers: shared hit=102
    -> Hash (cost=13.00..13.00 rows=600 width=19) (actual time=0.167..0.168
rows=600 loops=1)
      Buckets: 1024  Batches: 1  Memory Usage: 39kB
      Buffers: shared hit=7
    -> Seq Scan on products p (cost=0.00..13.00 rows=600 width=19)
(actual time=0.028..0.105 rows=600 loops=1)
      Buffers: shared hit=7

Planning:
  Buffers: shared hit=303
  Planning Time: 2.003 ms
  Execution Time: 15.623 ms

```


튜닝 포인트

변경 상품 매출 집계 → 순위 → 상품명 조회로 역할 분리

튜닝 후

```
WITH product_revenue AS (
  SELECT
    oi.product_id,
    sum(oi.line_total) AS revenue
  FROM ecom.order_items oi
  JOIN ecom.orders o
    ON o.order_id = oi.order_id
  WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  GROUP BY oi.product_id
),
ranked_products AS (
  SELECT
    product_id,
    revenue,
    rank() OVER (ORDER BY revenue DESC) AS revenue_rank
  FROM product_revenue
)

SELECT
  rp.product_id,
  p.product_name,
  rp.revenue,
  rp.revenue_rank
FROM ranked_products rp
JOIN ecom.products p
  ON p.product_id = rp.product_id
WHERE rp.revenue_rank <= 20
ORDER BY
  rp.revenue_rank,
  rp.product_id
LIMIT 20
```

튜닝 후 실행 결과

Q04 튜닝 후 결과

product_id	product_name	revenue	revenue_rank
457	Product 208	2,340,762.83	1
588	Product 510	998,376.91	2
85	Product 112	21,508.92	3
317	Product 156	17,799.75	4
347	Product 466	17,102.35	5
467	Product 58	16,838.52	6
234	Product 174	16,782.15	7
540	Product 359	15,995.89	8

... 12 rows omitted
rows=20 | median execution=4.087 ms

튜닝 후 성능 원문 로그 · Q04_plan_after.png (1/2)

```

Q04 상품별 누적매출 RANK Top 20 [after]

PERFORMANCE SQL
EXPLAIN (ANALYZE, BUFFERS)
WITH product_revenue AS (
    SELECT
        oi.product_id,
        sum(oi.line_total) AS revenue
    FROM ecom.order_items oi
    JOIN ecom.orders o
        ON o.order_id = oi.order_id
    WHERE o.order_status IN ('paid', 'shipped', 'delivered')
    GROUP BY oi.product_id
),
ranked_products AS (
    SELECT
        product_id,
        revenue,
        rank() OVER (ORDER BY revenue DESC) AS revenue_rank
    FROM product_revenue
)
SELECT
    rp.product_id,
    p.product_name,
    rp.revenue,
    rp.revenue_rank
FROM ranked_products rp
JOIN ecom.products p
    ON p.product_id = rp.product_id
WHERE rp.revenue_rank <= 20
ORDER BY
    rp.revenue_rank,
    rp.product_id
LIMIT 20

PERFORMANCE RESULT
Limit (cost=1115.04,1115.09 rows=20 width=59) (actual time=10.185..10.193 rows=20 loops=1)
  Buffers: shared hit=373
-> Sort (cost=1115.04,1116.54 rows=600 width=59) (actual time=10.184..10.187 rows=20 loops=1)
  Sort Key: (rank() OVER (?)), product_revenue.product_id
  Sort Method: quicksort Memory: 26kB
  Buffers: shared hit=373
-> Hash Join (cost=1087.00..1099.07 rows=600 width=59) (actual time=10.151..10.169 rows=20 loops=1)
  Hash Cond: (product_revenue.product_id = p.product_id)
  Buffers: shared hit=370
  -> WindowAgg (cost=1066.50..1076.98 rows=600 width=48) (actual time=10.014..10.026 rows=20 loops=1)
    Run Condition: (rank() OVER (?) <= 20)
    Buffers: shared hit=363
    -> Sort (cost=1066.48..1067.98 rows=600 width=40) (actual time=10.006..10.008 rows=21 loops=1)
      Sort Key: product_revenue.revenue DESC
      Sort Method: quicksort Memory: 44kB
      Buffers: shared hit=363
      -> Subquery Scan on product_revenue (cost=1031.30..1038.80 rows=600 width=40) (actual time=9.716..9.851 rows=600 loops=1)
        Buffers: shared hit=360
        -> HashAggregate (cost=1031.30..1038.80 rows=600 width=40) (actual time=9.715..9.805 rows=600 loops=1)
          Group Key: oi.product_id
          Batches: 1 Memory Usage: 297kB
          Buffers: shared hit=360
          -> Hash Join (cost=333.15..928.47 rows=20565 width=14) (actual time=9.247..9.422 rows=20565 loops=1)
            Join Key: (oi.order_id = o.order_id)
            Buffers: shared hit=359
            -> SeqScan on ecom.order_items oi (cost=0.00..1000000.00 rows=1000000 width=14) (actual time=0.000..0.000 rows=1000000 loops=1)
              Buffers: shared hit=1
            -> SeqScan on ecom.orders o (cost=0.00..1000000.00 rows=1000000 width=14) (actual time=0.000..0.000 rows=1000000 loops=1)
              Buffers: shared hit=1

```

튜닝 후 성능 원문 로그 · Q04_plan_after.png (2/2)

```

time=2.347..7.152 rows=20357 loops=1)
    Hash Cond: (oi.order_id = o.order_id)
    Buffers: shared hit=360
    -> Seq Scan on order_items oi (cost=0.00..525.16 rows=26716
width=22) (actual time=0.003..1.510 rows=26716 loops=1)
    Buffers: shared hit=258
    -> Hash (cost=237.99..237.99 rows=7613 width=8) (actual
time=2.329..2.330 rows=7613 loops=1)
    Buckets: 8192 Batches: 1 Memory Usage: 362kB
    Buffers: shared hit=102
    -> Seq Scan on orders o (cost=0.00..237.99 rows=7613
width=8) (actual time=0.005..1.572 rows=7613 loops=1)
    Filter: (order_status = ANY
('paid,shipped,delivered')::text[]))
    Rows Removed by Filter: 2277
    Buffers: shared hit=102
    -> Hash (cost=13.00..13.00 rows=600 width=19) (actual time=0.130..0.130 rows=600 loops=1)
    Buckets: 1024 Batches: 1 Memory Usage: 41kB
    Buffers: shared hit=7
    -> Seq Scan on products p (cost=0.00..13.00 rows=600 width=19) (actual
time=0.006..0.067 rows=600 loops=1)
    Buffers: shared hit=7
Planning:
  Buffers: shared hit=354
Planning Time: 1.153 ms
Execution Time: 10.315 ms

```

튜닝 전후 실행계획 비교

측정 항목	튜닝 전	튜닝 후
Execution Time	5.500 ms	4.087 ms
Planning Time	0.086 ms	0.073 ms
Actual Rows	20	20
Shared Hit	367	367
Shared Read	0	0
Top Plan Node	Limit	Limit
결과 동일	-	YES

업무 판단 및 요청 부서 전달안

튜닝 판단

정확히 20개를 반환하고 동점은 product_id로 결정했다.

MD 전달안

누적매출 1위는 Product 208(ID 457), 매출 2,340,762.83다. 결과는 정확히 20개로 고정했다.

Q5. 고객 RFM

요청 부서	CRM
요청 목적	고객별 최근성·빈도·금액 파악
판정 기준	유효 주문·주문 단위 빈도

튜닝 전

주문상품 행에서 COUNT DISTINCT로 빈도 복원

```
SELECT
  c.customer_id,
  c.full_name,
  current_date - max(o.order_ts)::date AS recency_days,
  count(DISTINCT o.order_id) AS frequency,
  sum(oi.line_total) AS monetary
FROM ecom.customers c
JOIN ecom.orders o
  ON o.customer_id = c.customer_id
JOIN ecom.order_items oi
  ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
GROUP BY
  c.customer_id,
  c.full_name
ORDER BY
  monetary DESC,
  c.customer_id
```

튜닝 전 실행 결과

Q05 튜닝 전 결과

customer_id	full_name	recency_days	frequency	monetary
19	Customer 19	0	26	37,803.69
7	Customer 7	5	23	36,318.34
3	Customer 3	2	26	33,809.53
25	Customer 25	3	26	33,689.05
1	Customer 1	2	26	33,226.61
10	Customer 10	0	26	33,080.74
21	Customer 21	0	26	32,305.26
15	Customer 15	7	26	31,575.76
... 2,242 rows omitted				
rows=2,250 median execution=9.093 ms				

튜닝 전 성능 원문 로그 · Q05_plan_before.png (1/2)

```

Q05 고객 RFM [before]

PERFORMANCE SQL
EXPLAIN (ANALYZE, BUFFERS)
SELECT
    c.customer_id,
    c.full_name,
    current_date - max(o.order_ts)::date AS recency_days,
    count(DISTINCT o.order_id) AS frequency,
    sum(oi.line_total) AS monetary
FROM ecom.customers c
JOIN ecom.orders o
    ON o.customer_id = c.customer_id
JOIN ecom.order_items oi
    ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
GROUP BY
    c.customer_id,
    c.full_name
ORDER BY
    monetary DESC,
    c.customer_id

PERFORMANCE RESULT
Sort (cost=3053.62..3061.12 rows=3000 width=65) (actual time=16.944..17.022 rows=2250 loops=1)
  Sort Key: (sum(oi.line_total)) DESC, c.customer_id
  Sort Method: quicksort  Memory: 237kB
  Buffers: shared hit=406
  -> GroupAggregate (cost=2563.30..2880.36 rows=3000 width=65) (actual time=13.422..16.460 rows=2250
loops=1)
    Group Key: c.customer_id
    Buffers: shared hit=400
    -> Sort (cost=2563.30..2614.71 rows=20565 width=43) (actual time=13.391..13.966 rows=20557
loops=1)
      Sort Key: c.customer_id, o.order_id
      Sort Method: quicksort  Memory: 2053kB
      Buffers: shared hit=400
      -> Hash Join (cost=440.65..1090.03 rows=20565 width=43) (actual time=1.915..9.393
rows=20557 loops=1)
        Hash Cond: (o.customer_id = c.customer_id)
        Buffers: shared hit=400
        -> Hash Join (cost=333.15..928.47 rows=20565 width=30) (actual time=1.399..6.478
rows=20557 loops=1)
          Hash Cond: (oi.order_id = o.order_id)
          Buffers: shared hit=360
          -> Seq Scan on order_items oi (cost=0.00..525.16 rows=26716 width=14) (actual
time=0.002..1.540 rows=26716 loops=1)
            Buffers: shared hit=258
            -> Hash (cost=237.99..237.99 rows=7613 width=24) (actual time=1.388..1.388
rows=7613 loops=1)
              Buckets: 8192  Batches: 1  Memory Usage: 481kB
              Buffers: shared hit=102
              -> Seq Scan on orders o (cost=0.00..237.99 rows=7613 width=24) (actual
time=0.002..0.967 rows=7613 loops=1)
                Filter: (order_status = ANY ('{paid,shipped,delivered}':text[]))
                Rows Removed by Filter: 2277
                Buffers: shared hit=102
                -> Hash (cost=70.00..70.00 rows=3000 width=21) (actual time=0.509..0.509 rows=3000
loops=1)
                  Buckets: 4096  Batches: 1  Memory Usage: 190kB
                  Buffers: shared hit=40
                  -> Seq Scan on customers c (cost=0.00..70.00 rows=3000 width=21) (actual
time=0.007..0.279 rows=3000 loops=1)
                    Buffers: shared hit=40
Planning:
  Buffers: shared hit=339

```

튜닝 전 성능 원문 로그 · Q05_plan_before.png (2/2)

```

  Buffers: shared hit=339
Planning Time: 1.202 ms
Execution Time: 17.166 ms

```

튜닝 포인트

변경 주문별 금액을 선집계한 뒤 고객 RFM 계산

튜닝 후

```

WITH order_totals AS (
  SELECT
    o.customer_id,
    o.order_id,
    o.order_ts,
    sum(oi.line_total) AS amount
  FROM ecom.orders o
  JOIN ecom.order_items oi
    ON oi.order_id = o.order_id
  WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  GROUP BY
    o.customer_id,
    o.order_id,
    o.order_ts
)

SELECT
  c.customer_id,
  c.full_name,
  current_date - max(ot.order_ts)::date AS recency_days,
  count(*) AS frequency,
  sum(ot.amount) AS monetary
FROM order_totals ot
JOIN ecom.customers c
  ON c.customer_id = ot.customer_id
GROUP BY
  c.customer_id,
  c.full_name
ORDER BY
  monetary DESC,
  c.customer_id

```

튜닝 후 실행 결과

Q05 튜닝 후 결과

customer_id	full_name	recency_days	frequency	monetary
19	Customer 19	0	26	37,803.69
7	Customer 7	5	23	36,318.34
3	Customer 3	2	26	33,809.53
25	Customer 25	3	26	33,689.05
1	Customer 1	2	26	33,226.61
10	Customer 10	0	26	33,080.74
21	Customer 21	0	26	32,305.26
15	Customer 15	7	26	31,575.76
... 2,242 rows omitted				
rows=2,250 median execution=7.129 ms				

튜닝 후 성능 원문 로그 · Q05_plan_after.png (1/2)

```

Q05 고객 RFM [after]

PERFORMANCE SQL
EXPLAIN (ANALYZE, BUFFERS)
WITH order_totals AS (
  SELECT
    o.customer_id,
    o.order_id,
    o.order_ts,
    sum(oi.line_total) AS amount
  FROM ecom.orders o
  JOIN ecom.order_items oi
    ON oi.order_id = o.order_id
  WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  GROUP BY
    o.customer_id,
    o.order_id,
    o.order_ts
)

SELECT
  c.customer_id,
  c.full_name,
  current_date - max(ot.order_ts)::date AS recency_days,
  count(*) AS frequency,
  sum(ot.amount) AS monetary
FROM order_totals ot
JOIN ecom.customers c
  ON c.customer_id = ot.customer_id
GROUP BY
  c.customer_id,
  c.full_name
ORDER BY
  monetary DESC,
  c.customer_id

PERFORMANCE RESULT
Sort (cost=1639.50..1647.00 rows=3000 width=65) (actual time=15.477..15.527 rows=2250 loops=1)
  Sort Key: (sum((sum(oi.line_total)))) DESC, c.customer_id
  Sort Method: quicksort  Memory: 237kB
  Buffers: shared hit=406
  -> HashAggregate (cost=1406.24..1466.24 rows=3000 width=65) (actual time=14.218..14.901 rows=2250
loops=1)
    Group Key: c.customer_id
    Batches: 1  Memory Usage: 1137kB
    Buffers: shared hit=400
    -> Hash Join (cost=1138.80..1330.11 rows=7613 width=61) (actual time=10.028..12.756 rows=7613
loops=1)
      Hash Cond: (o.customer_id = c.customer_id)
      Buffers: shared hit=400
      -> HashAggregate (cost=1031.30..1126.46 rows=7613 width=56) (actual time=9.419..10.999
rows=7613 loops=1)
        Group Key: o.order_id
        Batches: 1  Memory Usage: 3473kB
        Buffers: shared hit=360
        -> Hash Join (cost=333.15..928.47 rows=20565 width=30) (actual time=2.384..6.628
rows=20557 loops=1)
          Hash Cond: (oi.order_id = o.order_id)
          Buffers: shared hit=360
          -> Seq Scan on order_items oi (cost=0.00..525.16 rows=26716 width=14) (actual
time=0.003..1.193 rows=26716 loops=1)
            Buffers: shared hit=258
          -> Hash (cost=237.99..237.99 rows=7613 width=24) (actual time=2.360..2.361
rows=7613 loops=1)
            Buckets: 8192  Batches: 1  Memory Usage: 481kB
            Buffers: shared hit=163

```

튜닝 후 성능 원문 로그 · Q05_plan_after.png (2/2)

```

    buffers: shared hit=102
    -> Seq Scan on orders o (cost=0.00..237.99 rows=7613 width=24) (actual
time=0.005..1.554 rows=7613 loops=1)
      Filter: (order_status = ANY ('{paid,shipped,delivered} '::text[]))
      Rows Removed by Filter: 2277
      buffers: shared hit=102
    -> Hash (cost=70.00..70.00 rows=3000 width=21) (actual time=0.597..0.597 rows=3000
loops=1)
      Buckets: 4096 Batches: 1 Memory Usage: 190kB
      buffers: shared hit=40
    -> Seq Scan on customers c (cost=0.00..70.00 rows=3000 width=21) (actual
time=0.008..0.307 rows=3000 loops=1)
      buffers: shared hit=40
Planning:
  buffers: shared hit=326
  Planning Time: 1.106 ms
  Execution Time: 15.709 ms

```

튜닝 전후 실행계획 비교

측정 항목	튜닝 전	튜닝 후
Execution Time	9.093 ms	7.129 ms
Planning Time	0.090 ms	0.078 ms
Actual Rows	2,250	2,250
Shared Hit	400	400
Shared Read	0	0
Top Plan Node	Sort	Sort
결과 동일	-	YES

업무 판단 및 요청 부서 전달안

튜닝 판단
빈도와 금액의 집계 단위를 주문으로 통일했다.

CRM 전달안

최고 Monetary 고객은 Customer 19(ID 19)이며 구매 26회, 누적금액 37,803.69, 최근성 0일이다.

Q6. 첫 구매 후 30일 내 재구매율

요청 부서	CRM
요청 목적	첫 구매 후 30일 내 재구매율 확인
판정 기준	첫 주문 이후 추가 주문

튜닝 전

고객별 EXISTS 상관 서브쿼리를 반복

```
WITH purchases AS (
  SELECT
    customer_id,
    order_ts
  FROM ecom.orders
  WHERE order_status IN ('paid', 'shipped', 'delivered')
),
first_buy AS (
  SELECT
    customer_id,
    min(order_ts) AS first_order_ts
  FROM purchases
  GROUP BY customer_id
)

SELECT
  count(*) AS first_buyers,
  count(*) FILTER (
    WHERE EXISTS (
      SELECT 1
      FROM purchases p
      WHERE p.customer_id = f.customer_id
      AND p.order_ts > f.first_order_ts
      AND p.order_ts <= f.first_order_ts + interval '30 days'
    )
  ) AS repurchasers,
  ecom.safe_div(
    count(*) FILTER (
      WHERE EXISTS (
        SELECT 1
        FROM purchases p
        WHERE p.customer_id = f.customer_id
        AND p.order_ts > f.first_order_ts
        AND p.order_ts <= f.first_order_ts + interval '30 days'
      )
    ),
    count(*)
  ) AS repurchase_rate
FROM first_buy f
```

튜닝 전 실행 결과

Q06 튜닝 전 결과

```
first_buyers | repurchasers | repurchase_rate
-----+-----+-----
2250        | 1378        | 0.61
rows=1 | median execution=559.240 ms
```

튜닝 전 성능 원문 로그 · Q06_plan_before.png (1/2)

```

Q06 첫 구매 후 30일 내 재구매율 [before]

PERFORMANCE SQL
EXPLAIN (ANALYZE, BUFFERS)
WITH purchases AS (
  SELECT
    customer_id,
    order_ts
  FROM ecom.orders
  WHERE order_status IN ('paid', 'shipped', 'delivered')
),
first_buy AS (
  SELECT
    customer_id,
    min(order_ts) AS first_order_ts
  FROM purchases
  GROUP BY customer_id
)

SELECT
  count(*) AS first_buyers,
  count(*) FILTER (
    WHERE EXISTS (
      SELECT 1
      FROM purchases p
      WHERE p.customer_id = f.customer_id
      AND p.order_ts > f.first_order_ts
      AND p.order_ts <= f.first_order_ts + interval '30 days'
    )
  ) AS repurchasers,
  ecom.safe_div(
    count(*) FILTER (
      WHERE EXISTS (
        SELECT 1
        FROM purchases p
        WHERE p.customer_id = f.customer_id
        AND p.order_ts > f.first_order_ts
        AND p.order_ts <= f.first_order_ts + interval '30 days'
      )
    ),
    count(*)
  ) AS repurchase_rate
FROM first_buy f

PERFORMANCE RESULT
Aggregate (cost=1055173.74..1055173.76 rows=1 width=48) (actual time=710.017..710.021 rows=1 loops=1)
  Buffers: shared hit=102
  CTE purchases
    -> Seq Scan on orders (cost=0.00..237.99 rows=7613 width=16) (actual time=0.004..0.668 rows=7613 loops=1)
    Filter: (order_status = ANY ('(paid,shipped,delivered)::text[]))
    Rows Removed by Filter: 2277
    Buffers: shared hit=102
    -> HashAggregate (cost=190.32..213.41 rows=2309 width=16) (actual time=1.505..1.847 rows=2250 loops=1)
      Group Key: purchases.customer_id
      Batches: 1 Memory Usage: 369kB
      Buffers: shared hit=102
      -> CTE Scan on purchases (cost=0.00..152.26 rows=7613 width=16) (actual time=0.005..1.098 rows=7613 loops=1)
      Buffers: shared hit=102
    SubPlan 2
      -> CTE Scan on purchases p (cost=0.00..228.39 rows=1 width=0) (actual time=0.163..0.163 rows=1 loops=2250)
      Filter: ((order_ts > (min(purchases.order_ts))) AND (customer_id = purchases.customer_id) AND (order_ts <= ((min(purchases.order_ts)) + '30 days'::interval)))

```

튜닝 전 성능 원문 로그 · Q06_plan_before.png (2/2)

```

  (order_ts <= ((min(purchases.order_ts)) + '30 days'::interval)))
  Rows Removed by Filter: 4963
  SubPlan 3
    -> CTE Scan on purchases p_1 (cost=0.00..228.39 rows=1 width=0) (actual time=0.151..0.151 rows=1 loops=2250)
    Filter: (((order_ts > (min(purchases.order_ts))) AND (customer_id = purchases.customer_id) AND (order_ts <= ((min(purchases.order_ts)) + '30 days'::interval)))
    Rows Removed by Filter: 4963
  Planning:
    Buffers: shared hit=408
  Planning Time: 0.424 ms
  Execution Time: 710.099 ms

```

튜닝 포인트

변경 원도 함수로 첫 구매일 계산 후 **BOOL_OR**로 고객당 1회 집계

튜닝 후

```
WITH purchases AS (
  SELECT
    customer_id,
    order_ts,
    min(order_ts) OVER (
      PARTITION BY customer_id
    ) AS first_order_ts
  FROM ecom.orders
  WHERE order_status IN ('paid', 'shipped', 'delivered')
),
customer_flags AS (
  SELECT
    customer_id,
    bool_or(
      order_ts > first_order_ts
      AND order_ts <= first_order_ts + interval '30 days'
    ) AS repurchased
  FROM purchases
  GROUP BY customer_id
)

SELECT
  count(*) AS first_buyers,
  count(*) FILTER (WHERE repurchased) AS repurchasers,
  ecom.safe_div(
    count(*) FILTER (WHERE repurchased),
    count(*)
  ) AS repurchase_rate
FROM customer_flags
```

튜닝 후 실행 결과

Q06 튜닝 후 결과

first_buyers	repurchasers	repurchase_rate
2250	1378	0.61

rows=1 | median execution=2.750 ms

튜닝 후 성능 원문 로그 · Q06_plan_after.png (1/2)

```

Q06 첫 구매 후 30일 내 재구매율 [after]

PERFORMANCE SQL
EXPLAIN (ANALYZE, BUFFERS)
WITH purchases AS (
  SELECT
    customer_id,
    order_ts,
    min(order_ts) OVER (
      PARTITION BY customer_id
    ) AS first_order_ts
  FROM ecom.orders
  WHERE order_status IN ('paid', 'shipped', 'delivered')
),
customer_flags AS (
  SELECT
    customer_id,
    bool_or(
      order_ts > first_order_ts
      AND order_ts <= first_order_ts + interval '30 days'
    ) AS repurchased
  FROM purchases
  GROUP BY customer_id
)

SELECT
  count(*) AS first_buyers,
  count(*) FILTER (WHERE repurchased) AS repurchasers,
  ecom.safe_div(
    count(*) FILTER (WHERE repurchased),
    count(*)
  ) AS repurchase_rate
FROM customer_flags

PERFORMANCE RESULT
Aggregate  (cost=1014.92..1014.95 rows=1 width=48) (actual time=2.957..2.959 rows=1 loops=1)
  Buffers: shared hit=105
  -> GroupAggregate  (cost=728.84..980.29 rows=2309 width=9) (actual time=1.097..2.902 rows=2250
loops=1)
    Group Key: orders.customer_id
    Buffers: shared hit=105
    -> WindowAgg  (cost=728.84..862.03 rows=7613 width=24) (actual time=1.086..2.016 rows=7613
loops=1)
      Buffers: shared hit=105
      -> Sort  (cost=728.81..747.84 rows=7613 width=16) (actual time=1.080..1.210 rows=7613
loops=1)
        Sort Key: orders.customer_id
        Sort Method: quicksort  Memory: 430kB
        Buffers: shared hit=105
        -> Seq Scan on orders  (cost=0.00..237.99 rows=7613 width=16) (actual
time=0.002..0.650 rows=7613 loops=1)
          Filter: (order_status = ANY ('{paid,shipped,delivered}':text[]))
          Rows Removed by Filter: 2277
          Buffers: shared hit=102

Planning:
  Buffers: shared hit=418
  Planning Time: 0.288 ms
  Execution Time: 2.983 ms

```

튜닝 후 성능 원문 로그 · Q06_plan_after.png (2/2)

튜닝 전후 실행계획 비교

측정 항목	튜닝 전	튜닝 후
Execution Time	559.240 ms	2.750 ms
Planning Time	0.073 ms	0.041 ms
Actual Rows	1	1

Shared Hit	102	102
Shared Read	0	0
Top Plan Node	Aggregate	Aggregate
결과 동일	-	YES

업무 판단 및 요청 부서 전달안

튜닝 판단

반복 탐색을 단일 스캔·집계 흐름으로 바꾼 대표 개선 사례다.

CRM 전달안

첫 구매 고객 2,250명 중 1,378명이 30일 내 재구매해 재구매율은 61.2%다.

Q7. 재고 부족 위험 상품

요청 부서	구매·SCM
요청 목적	재주문이 필요한 상품 탐지
판정 기준	현재고 < 재주문 임계치

튜닝 전

상품마다 재고 스칼라 서브쿼리를 네 번 실행

```
SELECT
  p.product_id,
  p.product_name,
  (
    SELECT i.qty_on_hand
    FROM ecom.inventory i
    WHERE i.product_id = p.product_id
  ) AS qty_on_hand,
  (
    SELECT i.reorder_point
    FROM ecom.inventory i
    WHERE i.product_id = p.product_id
  ) AS reorder_point,
  (
    SELECT i.reorder_point - i.qty_on_hand
    FROM ecom.inventory i
    WHERE i.product_id = p.product_id
  ) AS shortage
FROM ecom.products p
WHERE (
  SELECT i.qty_on_hand < i.reorder_point
  FROM ecom.inventory i
  WHERE i.product_id = p.product_id
)
ORDER BY
  shortage DESC,
  p.product_id
```

튜닝 전 실행 결과

Q07 튜닝 전 결과

product_id	product_name	qty_on_hand	reorder_point	shortage
408	Product 37	0	30	30
427	Product 138	0	30	30
567	Product 20	0	30	30
165	Product 363	1	30	29
239	Product 84	1	30	29
378	Product 357	1	30	29
457	Product 208	1	30	29
547	Product 470	1	30	29
... 55 rows omitted				
rows=63 median execution=0.332 ms				

튜닝 전 성능 원문 로그 · Q07_plan_before.png (1/2)

```

Q07 재고 부족 위험 상품 [before]

PERFORMANCE SQL
EXPLAIN (ANALYZE, BUFFERS)
SELECT
  p.product_id,
  p.product_name,
  (
    SELECT i.qty_on_hand
    FROM ecom.inventory i
    WHERE i.product_id = p.product_id
  ) AS qty_on_hand,
  (
    SELECT i.reorder_point
    FROM ecom.inventory i
    WHERE i.product_id = p.product_id
  ) AS reorder_point,
  (
    SELECT i.reorder_point - i.qty_on_hand
    FROM ecom.inventory i
    WHERE i.product_id = p.product_id
  ) AS shortage
FROM ecom.products p
WHERE (
  SELECT i.qty_on_hand < i.reorder_point
  FROM ecom.inventory i
  WHERE i.product_id = p.product_id
)
ORDER BY
  shortage DESC,
  p.product_id

PERFORMANCE RESULT
Sort (cost=12466.34, 12467.09 rows=300 width=31) (actual time=0.908..0.913 rows=63 loops=1)
Sort Key: ((SubPlan 3)) DESC, p.product_id
Sort Method: quicksort Memory: 27kB
Buffers: shared hit=2380
-> Seq Scan on products p (cost=0.00..12454.00 rows=300 width=31) (actual time=0.038..0.872 rows=63 loops=1)
  Filter: (SubPlan 4)
  Rows Removed by Filter: 537
  Buffers: shared hit=2374
  SubPlan 1
    -> Index Scan using inventory_pkey on inventory i (cost=0.28..8.29 rows=1 width=4) (actual time=0.001..0.001 rows=1 loops=63)
    Index Cond: (product_id = p.product_id)
    Buffers: shared hit=189
  SubPlan 2
    -> Index Scan using inventory_pkey on inventory i_1 (cost=0.28..8.29 rows=1 width=4) (actual time=0.001..0.001 rows=1 loops=63)
    Index Cond: (product_id = p.product_id)
    Buffers: shared hit=189
  SubPlan 3
    -> Index Scan using inventory_pkey on inventory i_2 (cost=0.28..8.29 rows=1 width=4) (actual time=0.001..0.001 rows=1 loops=63)
    Index Cond: (product_id = p.product_id)
    Buffers: shared hit=189
  SubPlan 4
    -> Index Scan using inventory_pkey on inventory i_3 (cost=0.28..8.29 rows=1 width=1) (actual time=0.001..0.001 rows=1 loops=600)
    Index Cond: (product_id = p.product_id)
    Buffers: shared hit=1800
Planning:
  Buffers: shared hit=171
Planning Time: 0.637 ms
Execution Time: 0.964 ms

```

튜닝 전 성능 원문 로그 · Q07_plan_before.png (2/2)

Execution Time: 0.964 ms

튜닝 포인트

변경 inventory와 products를 한 번 JOIN

튜닝 후

```

SELECT
  p.product_id,
  p.product_name,
  i.qty_on_hand,
  i.reorder_point,
  i.reorder_point - i.qty_on_hand AS shortage
FROM ecom.inventory i
JOIN ecom.products p
  ON p.product_id = i.product_id
WHERE i.qty_on_hand < i.reorder_point
ORDER BY
  shortage DESC,
  p.product_id

```

튜닝 후 실행 결과

Q07 튜닝 후 결과

product_id	product_name	qty_on_hand	reorder_point	shortage
408	Product 37	0	30	30
427	Product 138	0	30	30
567	Product 20	0	30	30
165	Product 363	1	30	29
239	Product 84	1	30	29
378	Product 357	1	30	29
457	Product 208	1	30	29
547	Product 470	1	30	29
... 55 rows omitted				
rows=63 median execution=0.073 ms				

튜닝 후 성능 원문 로그 · Q07_plan_after.png

```

Q07 재고 부족 위험 상품 [after]

PERFORMANCE SQL
EXPLAIN (ANALYZE, BUFFERS)
SELECT
    p.product_id,
    p.product_name,
    i.qty_on_hand,
    i.reorder_point,
    i.reorder_point - i.qty_on_hand AS shortage
FROM ecom.inventory i
JOIN ecom.products p
    ON p.product_id = i.product_id
WHERE i.qty_on_hand < i.reorder_point
ORDER BY
    shortage DESC,
    p.product_id

PERFORMANCE RESULT
Sort (cost=37.73..38.23 rows=200 width=31) (actual time=0.297..0.305 rows=63 loops=1)
  Sort Key: ((i.reorder_point - i.qty_on_hand)) DESC, p.product_id
  Sort Method: quicksort  Memory: 27kB
  Buffers: shared hit=18
  -> Hash Join (cost=15.00..30.09 rows=200 width=31) (actual time=0.125..0.254 rows=63 loops=1)
    Hash Cond: (p.product_id = i.product_id)
    Buffers: shared hit=12
    -> Seq Scan on products p (cost=0.00..13.00 rows=600 width=19) (actual time=0.007..0.064
rows=600 loops=1)
      Buffers: shared hit=7
    -> Hash (cost=12.50..12.50 rows=200 width=16) (actual time=0.108..0.108 rows=63 loops=1)
      Buckets: 1024 Batches: 1 Memory Usage: 11kB
      Buffers: shared hit=5
      -> Seq Scan on inventory i (cost=0.00..12.50 rows=200 width=16) (actual time=0.014..0.086
rows=63 loops=1)
        Filter: (qty_on_hand < reorder_point)
        Rows Removed by Filter: 537
        Buffers: shared hit=5
Planning:
  Buffers: shared hit=199
Planning Time: 1.128 ms
Execution Time: 0.368 ms

```

튜닝 전후 실행계획 비교

측정 항목	튜닝 전	튜닝 후
Execution Time	0.332 ms	0.073 ms
Planning Time	0.029 ms	0.035 ms
Actual Rows	63	63
Shared Hit	2,374	12
Shared Read	0	0
Top Plan Node	Sort	Sort
결과 동일	-	YES

업무 판단 및 요청 부서 전달안

튜닝 판단

같은 테이블 반복 조회를 제거했다.

구매·SCM 전달안

재고 임계치 미달 상품은 63개다. 가장 긴급한 Product 37(ID 408)은 현재고 0, 임계치 30, 부족분 30다.

Q8. 리뷰 기반 효자상품

요청 부서	MD·마케팅
요청 목적	리뷰 기반 효자상품 선별
판정 기준	평점 4.5 이상·리뷰 50개 이상

튜닝 전

상품명까지 조인한 뒤 모든 리뷰를 집계

```
SELECT
  p.product_id,
  p.product_name,
  avg(r.rating) AS avg_rating,
  count(*) AS review_count
FROM ecom.products p
JOIN ecom.reviews r
  ON r.product_id = p.product_id
GROUP BY
  p.product_id,
  p.product_name
HAVING avg(r.rating) >= 4.5
AND count(*) >= 50
ORDER BY
  avg_rating DESC,
  review_count DESC,
  p.product_id
```

튜닝 전 실행 결과

Q08 튜닝 전 결과

product_id	product_name	avg_rating	review_count
418	Product 297	4.86	160
544	Product 260	4.81	161
166	Product 83	4.80	160
267	Product 235	4.79	160
490	Product 259	4.79	160
552	Product 60	4.78	161
341	Product 406	4.77	160
468	Product 268	4.76	160
... 4 rows omitted			

rows=12 | median execution=0.432 ms

튜닝 전 성능 원문 로그 · Q08_plan_before.png

```

Q08 리뷰 기반 효자상품 [before]

PERFORMANCE SQL
EXPLAIN (ANALYZE, BUFFERS)
SELECT
    p.product_id,
    p.product_name,
    avg(r.rating) AS avg_rating,
    count(*) AS review_count
FROM ecom.products p
JOIN ecom.reviews r
    ON r.product_id = p.product_id
GROUP BY
    p.product_id,
    p.product_name
HAVING avg(r.rating) >= 4.5
    AND count(*) >= 50
ORDER BY
    avg_rating DESC,
    review_count DESC,
    p.product_id

PERFORMANCE RESULT
Sort (cost=95.24..95.41 rows=67 width=59) (actual time=1.308..1.313 rows=12 loops=1)
  Sort Key: (avg(r.rating)) DESC, (count(*)) DESC, p.product_id
  Sort Method: quicksort  Memory: 25kB
  Buffers: shared hit=34
-> HashAggregate (cost=82.71..93.21 rows=67 width=59) (actual time=1.229..1.274 rows=12 loops=1)
  Group Key: p.product_id
  Filter: ((avg(r.rating) >= 4.5) AND (count(*) >= 50))
  Batches: 1  Memory Usage: 73kB
  Rows Removed by Filter: 123
  Buffers: shared hit=28
-> Hash Join (cost=20.50..67.36 rows=2046 width=23) (actual time=0.198..0.825 rows=2046
loops=1)
  Hash Cond: (r.product_id = p.product_id)
  Buffers: shared hit=28
-> Seq Scan on reviews r (cost=0.00..41.46 rows=2046 width=12) (actual time=0.004..0.171
rows=2046 loops=1)
  Buffers: shared hit=21
-> Hash (cost=13.00..13.00 rows=600 width=19) (actual time=0.187..0.187 rows=600 loops=1)
  Buckets: 1024  Batches: 1  Memory Usage: 39kB
  Buffers: shared hit=7
-> Seq Scan on products p (cost=0.00..13.00 rows=600 width=19) (actual
time=0.004..0.090 rows=600 loops=1)
  Buffers: shared hit=7

Planning:
  Buffers: shared hit=228
Planning Time: 1.117 ms
Execution Time: 1.387 ms

```

튜닝 포인트

변경 리뷰를 상품별로 먼저 집계·필터 후 상품명 조회

튜닝 후

```

WITH review_summary AS (
    SELECT
        product_id,
        avg(rating) AS avg_rating,
        count(*) AS review_count
    FROM ecom.reviews
    GROUP BY product_id
    HAVING avg(rating) >= 4.5
        AND count(*) >= 50
)

SELECT
    rs.product_id,
    p.product_name,
    rs.avg_rating,
    rs.review_count
FROM review_summary rs
JOIN ecom.products p

```

```
ON p.product_id = rs.product_id
ORDER BY
  rs.avg_rating DESC,
  rs.review_count DESC,
  rs.product_id
```

튜닝 후 실행 결과

Q08 튜닝 후 결과

product_id	product_name	avg_rating	review_count
418	Product 297	4.86	160
544	Product 260	4.81	161
166	Product 83	4.80	160
267	Product 235	4.79	160
490	Product 259	4.79	160
552	Product 60	4.78	161
341	Product 406	4.77	160
468	Product 268	4.76	160
... 4 rows omitted			
rows=12 median execution=0.270 ms			

튜닝 후 성능 원문 로그 · Q08_plan_after.png (1/2)

```

Q08 리뷰 기반 효자상품 [after]

PERFORMANCE SQL
EXPLAIN (ANALYZE, BUFFERS)
WITH review_summary AS (
  SELECT
    product_id,
    avg(rating) AS avg_rating,
    count(*) AS review_count
  FROM ecom.reviews
  GROUP BY product_id
  HAVING avg(rating) >= 4.5
  AND count(*) >= 50
)

SELECT
  rs.product_id,
  p.product_name,
  rs.avg_rating,
  rs.review_count
FROM review_summary rs
JOIN ecom.products p
  ON p.product_id = rs.product_id
ORDER BY
  rs.avg_rating DESC,
  rs.review_count DESC,
  rs.product_id

PERFORMANCE RESULT
Sort (cost=74.23..74.27 rows=15 width=59) (actual time=1.023..1.028 rows=12 loops=1)
  Sort Key: (avg(reviews.rating)) DESC, (count(*)) DESC, reviews.product_id
  Sort Method: quicksort  Memory: 25kB
  Buffers: shared hit=34
-> Hash Join (cost=59.35..73.94 rows=15 width=59) (actual time=0.854..0.983 rows=12 loops=1)
  Hash Cond: (p.product_id = reviews.product_id)
  Buffers: shared hit=28
-> Seq Scan on products p (cost=0.00..13.00 rows=600 width=19) (actual time=0.008..0.076
rows=600 loops=1)
  Buffers: shared hit=7
-> Hash (cost=59.17..59.17 rows=15 width=48) (actual time=0.834..0.834 rows=12 loops=1)
  Buckets: 1024 Batches: 1 Memory Usage: 9kB
  Buffers: shared hit=21
-> HashAggregate (cost=56.80..59.17 rows=15 width=48) (actual time=0.775..0.824 rows=12
loops=1)
  Group Key: reviews.product_id
  Filter: ((avg(reviews.rating) >= 4.5) AND (count(*) >= 50))
  Batches: 1 Memory Usage: 64kB
  Rows Removed by Filter: 123
  Buffers: shared hit=21
-> Seq Scan on reviews (cost=0.00..41.46 rows=2046 width=12) (actual
time=0.005..0.238 rows=2046 loops=1)
  Buffers: shared hit=21

Planning:
  Buffers: shared hit=209
  Planning Time: 0.974 ms
  Execution Time: 1.180 ms

```

튜닝 후 성능 원문 로그 · Q08_plan_after.png (2/2)

튜닝 전후 실행계획 비교

측정 항목	튜닝 전	튜닝 후
Execution Time	0.432 ms	0.270 ms
Planning Time	0.042 ms	0.034 ms
Actual Rows	12	12
Shared Hit	28	28

Shared Read	0	0
Top Plan Node	Sort	Sort
결과 동일	-	YES

업무 판단 및 요청 부서 전달안

튜닝 판단

효자 후보만 마스터 데이터와 결합한다.

MD·마케팅 전달안

효자상품 조건을 충족한 상품은 12개다. 상위 Product 297(ID 418)은 평점 4.86, 리뷰 160개다.

Q9. 쿠폰 사용 여부별 평균 주문 금액

요청 부서	마케팅·재무
요청 목적	쿠폰 사용 여부별 AOV 비교
판정 기준	주문별 금액을 먼저 집계

튜닝 전

주문상품 행 집계÷COUNT DISTINCT로 AOV 계산

```
SELECT
  o.coupon_code IS NOT NULL AS used_coupon,
  count(DISTINCT o.order_id) AS order_count,
  sum(oi.line_total) / count(DISTINCT o.order_id) AS avg_order_amount
FROM ecom.orders o
JOIN ecom.order_items oi
  ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
GROUP BY o.coupon_code IS NOT NULL
ORDER BY used_coupon
```

튜닝 전 실행 결과

Q09 튜닝 전 결과

```
used_coupon | order_count | avg_order_amount
-----+-----+-----
False      | 5796       | 628.88
True       | 1817       | 1,839.69
rows=2 | median execution=8.215 ms
```


튜닝 전 성능 원문 로그 · Q09_plan_before.png

```

Q09 쿠폰 사용 여부별 평균 주문 금액 [before]

PERFORMANCE SQL
EXPLAIN (ANALYZE, BUFFERS)
SELECT
    o.coupon_code IS NOT NULL AS used_coupon,
    count(DISTINCT o.order_id) AS order_count,
    sum(oi.line_total) / count(DISTINCT o.order_id) AS avg_order_amount
FROM ecom.orders o
JOIN ecom.order_items oi
    ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
GROUP BY o.coupon_code IS NOT NULL
ORDER BY used_coupon

PERFORMANCE RESULT
GroupAggregate (cost=2401.74..2607.42 rows=2 width=41) (actual time=7.779..8.156 rows=2 loops=1)
  Group Key: ((o.coupon_code IS NOT NULL))
  Buffers: shared hit=368
  -> Sort (cost=2401.74..2453.15 rows=20565 width=15) (actual time=6.905..7.259 rows=20557 loops=1)
    Sort Key: ((o.coupon_code IS NOT NULL)), o.order_id
    Sort Method: quicksort  Memory: 1572kB
    Buffers: shared hit=368
    -> Hash Join (cost=333.15..928.47 rows=20565 width=15) (actual time=1.268..3.837 rows=20557
loops=1)
      Hash Cond: (oi.order_id = o.order_id)
      Buffers: shared hit=360
      -> Seq Scan on order_items oi (cost=0.00..525.16 rows=26716 width=14) (actual
time=0.002..0.785 rows=26716 loops=1)
        Buffers: shared hit=258
      -> Hash (cost=237.99..237.99 rows=7613 width=15) (actual time=1.253..1.254 rows=7613
loops=1)
        Buckets: 8192  Batches: 1  Memory Usage: 376kB
        Buffers: shared hit=102
        -> Seq Scan on orders o (cost=0.00..237.99 rows=7613 width=15) (actual
time=0.003..0.878 rows=7613 loops=1)
          Filter: (order_status = ANY ('{paid,shipped,delivered}':text[]))
          Rows Removed by Filter: 2277
          Buffers: shared hit=102
Planning:
  Buffers: shared hit=276
Planning Time: 0.616 ms
Execution Time: 8.224 ms

```

튜닝 포인트

변경 주문별 금액 선집계 후 쿠폰 그룹 AVG

튜닝 후

```

WITH order_totals AS (
    SELECT
        o.order_id,
        o.coupon_code IS NOT NULL AS used_coupon,
        sum(oi.line_total) AS amount
    FROM ecom.orders o
    JOIN ecom.order_items oi
        ON oi.order_id = o.order_id
    WHERE o.order_status IN ('paid', 'shipped', 'delivered')
    GROUP BY
        o.order_id,
        o.coupon_code IS NOT NULL
)

SELECT
    used_coupon,
    count(*) AS order_count,
    avg(amount) AS avg_order_amount
FROM order_totals
GROUP BY used_coupon
ORDER BY used_coupon

```

튜닝 후 실행 결과

Q09 튜닝 후 결과

used_coupon	order_count	avg_order_amount
False	5796	628.88
True	1817	1,839.69

rows=2 | median execution=5.571 ms

튜닝 후 성능 원문 로그 · Q09_plan_after.png (1/2)

```
Q09 쿠폰 사용 여부별 평균 주문 금액 [after]

PERFORMANCE SQL
EXPLAIN (ANALYZE, BUFFERS)
WITH order_totals AS (
  SELECT
    o.order_id,
    o.coupon_code IS NOT NULL AS used_coupon,
    sum(oi.line_total) AS amount
  FROM ecom.orders o
  JOIN ecom.order_items oi
    ON oi.order_id = o.order_id
  WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  GROUP BY
    o.order_id,
    o.coupon_code IS NOT NULL
)

SELECT
  used_coupon,
  count(*) AS order_count,
  avg(amount) AS avg_order_amount
FROM order_totals
GROUP BY used_coupon
ORDER BY used_coupon

PERFORMANCE RESULT
Sort (cost=1539.53..1539.53 rows=2 width=41) (actual time=8.816..8.819 rows=2 loops=1)
  Sort Key: ((o.coupon_code IS NOT NULL))
  Sort Method: quicksort  Memory: 25kB
  Buffers: shared hit=365
  -> HashAggregate (cost=1539.49..1539.52 rows=2 width=41) (actual time=8.803..8.804 rows=2 loops=1)
    Group Key: ((o.coupon_code IS NOT NULL))
    Batches: 1  Memory Usage: 24kB
    Buffers: shared hit=360
    -> HashAggregate (cost=1082.71..1273.04 rows=15226 width=41) (actual time=7.464..8.391
rows=7613 loops=1)
      Group Key: o.order_id, (o.coupon_code IS NOT NULL)
      Batches: 1  Memory Usage: 3601kB
      Buffers: shared hit=360
      -> Hash Join (cost=333.15..928.47 rows=20565 width=15) (actual time=1.657..5.164
rows=20557 loops=1)
        Hash Cond: (oi.order_id = o.order_id)
        Buffers: shared hit=360
        -> Seq Scan on order_items oi (cost=0.00..525.16 rows=26716 width=14) (actual
time=0.003..1.023 rows=26716 loops=1)
          Buffers: shared hit=258
        -> Hash (cost=237.99..237.99 rows=7613 width=15) (actual time=1.643..1.643
rows=7613 loops=1)
          Buckets: 8192  Batches: 1  Memory Usage: 374kB
          Buffers: shared hit=102
          -> Seq Scan on orders o (cost=0.00..237.99 rows=7613 width=15) (actual
time=0.004..1.148 rows=7613 loops=1)
            Filter: (order_status = ANY ('{paid,shipped,delivered} '::text[]))
            Rows Removed by Filter: 2277
            Buffers: shared hit=102

Planning:
  Buffers: shared hit=279
Planning Time: 0.415 ms
Execution Time: 8.916 ms
```

튜닝 후 성능 원문 로그 · Q09_plan_after.png (2/2)

튜닝 전후 실행계획 비교

측정 항목	튜닝 전	튜닝 후
Execution Time	8.215 ms	5.571 ms
Planning Time	0.065 ms	0.047 ms
Actual Rows	2	2
Shared Hit	360	360
Shared Read	0	0
Top Plan Node	Aggregate	Sort
결과 동일	-	YES

업무 판단 및 요청 부서 전달안

튜닝 판단

AOV의 업무 단위가 주문임을 SQL 구조에 직접 반영했다.

마케팅·재무 전달안

쿠폰 사용 주문 AOV는 1,839.69, 미사용 주문은 628.88로 표본상 192.5% 높다. 인과효과가 아니라 관찰 비교다.

Q10. 상위 1% 고객의 최근 60일 매출

요청 부서	CRM·VIP
요청 목적	상위 1% 고객의 최근 60일 매출
판정 기준	누적매출 Top 30 후 60일 집계

튜닝 전

Top 30 고객마다 최근 60일 매출 상관 조회

```
WITH lifetime AS (
  SELECT
    o.customer_id,
    sum(oi.line_total) AS lifetime_revenue
  FROM ecom.orders o
  JOIN ecom.order_items oi
    ON oi.order_id = o.order_id
  WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  GROUP BY o.customer_id
),
top_customers AS (
  SELECT
    customer_id,
    lifetime_revenue
  FROM lifetime
  ORDER BY
    lifetime_revenue DESC,
    customer_id
  LIMIT 30
)

SELECT
  tc.customer_id,
  tc.lifetime_revenue,
  (
    SELECT sum(oi.line_total)
    FROM ecom.orders o
    JOIN ecom.order_items oi
      ON oi.order_id = o.order_id
    WHERE o.customer_id = tc.customer_id
      AND o.order_status IN ('paid', 'shipped', 'delivered')
      AND o.order_ts >= now() - interval '60 days'
  ) AS recent_60d_revenue
FROM top_customers tc
ORDER BY
  tc.lifetime_revenue DESC,
  tc.customer_id
```

튜닝 전 실행 결과

Q10 튜닝 전 결과

customer_id	lifetime_revenue	recent_60d_revenue
19	37,803.69	37,803.69
7	36,318.34	36,318.34
3	33,809.53	32,603.95
25	33,689.05	32,619.33
1	33,226.61	30,255.84
10	33,080.74	32,851.70
21	32,305.26	31,585.52
15	31,575.76	30,963.58
... 22 rows omitted		
rows=30 median execution=5.006 ms		

튜닝 전 성능 원문 로그 · Q10_plan_before.png (1/2)

```

Q10 상위 1% 고객의 최근 60일 매출 [before]

PERFORMANCE SQL
EXPLAIN (ANALYZE, BUFFERS)
WITH lifetime AS (
  SELECT
    o.customer_id,
    sum(oi.line_total) AS lifetime_revenue
  FROM ecom.orders o
  JOIN ecom.order_items oi
    ON oi.order_id = o.order_id
  WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  GROUP BY o.customer_id
),
top_customers AS (
  SELECT
    customer_id,
    lifetime_revenue
  FROM lifetime
  ORDER BY
    lifetime_revenue DESC,
    customer_id
  LIMIT 30
)

SELECT
  tc.customer_id,
  tc.lifetime_revenue,
  (
    SELECT sum(oi.line_total)
    FROM ecom.orders o
    JOIN ecom.order_items oi
      ON oi.order_id = o.order_id
    WHERE o.customer_id = tc.customer_id
      AND o.order_status IN ('paid', 'shipped', 'delivered')
      AND o.order_ts >= now() - interval '60 days'
  ) AS recent_60d_revenue
FROM top_customers tc
ORDER BY
  tc.lifetime_revenue DESC,
  tc.customer_id

PERFORMANCE RESULT
Subquery Scan on tc (cost=1156.39..1753.69 rows=30 width=72) (actual time=7.278..8.608 rows=30 loops=1)
  Buffers: shared hit=2865
-> Limit (cost=1156.39..1156.47 rows=30 width=40) (actual time=7.186..7.190 rows=30 loops=1)
  Buffers: shared hit=366
-> Sort (cost=1156.39..1163.83 rows=2976 width=40) (actual time=7.185..7.187 rows=30 loops=1)
  Sort Key: (sum(oi.line_total)) DESC, o.customer_id
  Sort Method: top-N heapsort Memory: 27kB
  Buffers: shared hit=366
-> HashAggregate (cost=1031.30..1068.50 rows=2976 width=40) (actual time=6.552..6.902
rows=2250 loops=1)
  Group Key: o.customer_id
  Batches: 1 Memory Usage: 1137kB
  Buffers: shared hit=360
-> Hash Join (cost=333.15..928.47 rows=20565 width=14) (actual time=1.284..4.740
rows=20557 loops=1)
  Hash Cond: (oi.order_id = o.order_id)
  Buffers: shared hit=360
-> Seq Scan on order_items oi (cost=0.00..525.16 rows=26716 width=14) (actual
time=0.004..1.039 rows=26716 loops=1)
  Buffers: shared hit=258
-> Hash (cost=237.99..237.99 rows=7613 width=16) (actual time=1.265..1.265
rows=7613 loops=1)
  Buffers: shared hit=258

```

튜닝 전 성능 원문 로그 · Q10_plan_before.png (2/2)

```

        Buckets: 8192 Batches: 1 Memory Usage: 421kB
        Buffers: shared hit=102
        -> Seq Scan on orders o (cost=0.00..237.99 rows=7613 width=16) (actual
time=0.004..0.879 rows=7613 loops=1)
            Filter: (order_status = ANY ('{paid,shipped,delivered}':text[]))
            Rows Removed by Filter: 2277
            Buffers: shared hit=102

SubPlan 1
    -> Aggregate (cost=19.90..19.91 rows=1 width=32) (actual time=0.047..0.047 rows=1 loops=30)
        Buffers: shared hit=2499
    -> Nested Loop (cost=4.60..19.89 rows=3 width=6) (actual time=0.006..0.042 rows=68 loops=30)
        Buffers: shared hit=2499
    -> Bitmap Heap Scan on orders o_1 (cost=4.31..11.52 rows=1 width=8) (actual
time=0.004..0.010 rows=24 loops=30)
        Recheck Cond: ((customer_id = tc.customer_id) AND (order_ts >= (now() - '60
days'::interval)))
        Filter: (order_status = ANY ('{paid,shipped,delivered}':text[]))
        Rows Removed by Filter: 0
        Heap Blocks: exact=279
        Buffers: shared hit=339
    -> Bitmap Index Scan on idx_orders_customer_ts (cost=0.00..4.31 rows=2 width=0)
(actual time=0.003..0.003 rows=24 loops=30)
        Index Cond: ((customer_id = tc.customer_id) AND (order_ts >= (now() - '60
days'::interval)))
        Buffers: shared hit=60
    -> Index Scan using idx_order_items_order on order_items oi_1 (cost=0.29..8.34 rows=3
width=14) (actual time=0.001..0.001 rows=3 loops=717)
        Index Cond: (order_id = o_1.order_id)
        Buffers: shared hit=2160

Planning:
    Buffers: shared hit=311
    Planning Time: 1.145 ms
    Execution Time: 8.698 ms

```

튜닝 포인트

변경 한 번의 고객 집계에서 누적 60일 매출을 조건부 합산

튜닝 후

```

WITH customer_sales AS (
    SELECT
        o.customer_id,
        sum(oi.line_total) AS lifetime_revenue,
        sum(oi.line_total) FILTER (
            WHERE o.order_ts >= now() - interval '60 days'
        ) AS recent_60d_revenue
    FROM ecom.orders o
    JOIN ecom.order_items oi
        ON oi.order_id = o.order_id
    WHERE o.order_status IN ('paid', 'shipped', 'delivered')
    GROUP BY o.customer_id
)

SELECT
    customer_id,
    lifetime_revenue,
    recent_60d_revenue
FROM customer_sales
ORDER BY
    lifetime_revenue DESC,
    customer_id
LIMIT 30

```

튜닝 후 실행 결과

Q10 튜닝 후 결과

customer_id	lifetime_revenue	recent_60d_revenue
19	37,803.69	37,803.69
7	36,318.34	36,318.34
3	33,809.53	32,603.95
25	33,689.05	32,619.33
1	33,226.61	30,255.84
10	33,080.74	32,851.70
21	32,305.26	31,585.52
15	31,575.76	30,963.58
... 22 rows omitted		
rows=30 median execution=6.807 ms		

튜닝 후 성능 원문 로그 · Q10_plan_after.png (1/2)

```
Q10 상위 1% 고객의 최근 60일 매출 [after]

PERFORMANCE SQL
EXPLAIN (ANALYZE, BUFFERS)
WITH customer_sales AS (
  SELECT
    o.customer_id,
    sum(o.line_total) AS lifetime_revenue,
    sum(o.line_total) FILTER (
      WHERE o.order_ts >= now() - interval '60 days'
    ) AS recent_60d_revenue
  FROM ecom.orders o
  JOIN ecom.order_items oi
    ON oi.order_id = o.order_id
  WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  GROUP BY o.customer_id
)

SELECT
  customer_id,
  lifetime_revenue,
  recent_60d_revenue
FROM customer_sales
ORDER BY
  lifetime_revenue DESC,
  customer_id
LIMIT 30

PERFORMANCE RESULT
Limit (cost=1369.48..1369.56 rows=30 width=72) (actual time=12.920..12.928 rows=30 loops=1)
  Buffers: shared hit=366
-> Sort (cost=1369.48..1376.92 rows=2976 width=72) (actual time=12.919..12.921 rows=30 loops=1)
  Sort Key: (sum(o.line_total)) DESC, o.customer_id
  Sort Method: top-N heapsort  Memory: 28kB
  Buffers: shared hit=366
-> HashAggregate (cost=1236.95..1281.59 rows=2976 width=72) (actual time=12.113..12.627
rows=2250 loops=1)
  Group Key: o.customer_id
  Batches: 1 Memory Usage: 1649kB
  Buffers: shared hit=360
-> Hash Join (cost=333.15..928.47 rows=20565 width=22) (actual time=2.294..6.494
rows=20557 loops=1)
  Hash Cond: (oi.order_id = o.order_id)
  Buffers: shared hit=360
-> Seq Scan on order_items oi (cost=0.00..525.16 rows=26716 width=14) (actual
time=0.006..1.208 rows=26716 loops=1)
  Buffers: shared hit=258
-> Hash (cost=237.99..237.99 rows=7613 width=24) (actual time=2.272..2.272
rows=7613 loops=1)
  Buckets: 8192 Batches: 1 Memory Usage: 481kB
  Buffers: shared hit=102
-> Seq Scan on orders o (cost=0.00..237.99 rows=7613 width=24) (actual
time=0.004..1.527 rows=7613 loops=1)
  Filter: (order_status = ANY ('{paid,shipped,delivered} '::text[]))
  Rows Removed by Filter: 2277
  Buffers: shared hit=102

Planning:
  Buffers: shared hit=289
Planning Time: 0.833 ms
Execution Time: 12.991 ms
```

튜닝 후 성능 원문 로그 · Q10_plan_after.png (2/2)



튜닝 전후 실행계획 비교

측정 항목	튜닝 전	튜닝 후
Execution Time	5.006 ms	6.807 ms
Planning Time	0.102 ms	0.046 ms
Actual Rows	30	30
Shared Hit	2,859	360
Shared Read	0	0
Top Plan Node	Subquery Scan	Limit
결과 동일	-	YES

업무 판단 및 요청 부서 전달안

튜닝 판단

고객당 반복 스캔을 한 번의 집계로 줄였다.

CRM·VIP 전달안

상위 1%는 30명으로 고정했다. 1위 고객 ID 19의 누적매출은 37,803.69, 최근 60일 매출은 37,803.69이며 Top 30의 최근 매출 합계는 822,357.89다.

Q11. 안전한 나눗셈 함수 비교

검증 목적	분모 0·NULL 처리 차이 확인
비교 함수	ecom.f_safe_div · ecom.safe_div
적용 판단	평균 계산은 관측 없음과 실제 0을 구분

검증 SQL 및 결과

```
SELECT
  ecom.f_safe_div(10, 2) AS f_normal,
  ecom.f_safe_div(10, 0) AS f_zero,
  ecom.safe_div(10, 2) AS safe_normal,
  ecom.safe_div(10, 0) AS safe_zero,
  ecom.safe_div(10, NULL) AS safe_null
```

Q11 안전한 나눗셈 함수 비교

SQL

```
SELECT
  ecom.f_safe_div(10, 2) AS f_normal,
  ecom.f_safe_div(10, 0) AS f_zero,
  ecom.safe_div(10, 2) AS safe_normal,
  ecom.safe_div(10, 0) AS safe_zero,
  ecom.safe_div(10, NULL) AS safe_null
```

RESULT

f_normal	f_zero	safe_normal	safe_zero	safe_null
5.0000000000000000	0	5.0000000000000000	NULL	NULL

total rows: 1

결과 정상 분모에서는 두 함수 모두 5를 반환했다. 분모가 0이면 f_safe_div는 0, safe_div는 NULL을 반환하며 safe_div는 NULL 분모도 NULL로 유지한다.

선택 평균값에서 ‘관측 없음’과 실제 0을 구분하기 위해 Q2·Q6에는 ecom.safe_div를 사용했다.

부록 A. 조인 방식 비교

같은 고객별 매출 집계 쿼리에서 PostgreSQL의 조인 옵션을 각각 강제해 실제 실행계획을 비교했다. 수치는 현재 시드 데이터와 캐시 상태에서의 관측값이다.

방식	실행 시간	Buffers hit	적합한 상황	이번 결과
Nested Loop	19.833 ms	23,069	외부 결과가 작고 내부 인덱스가 유효	orders 7,613행마다 order_items 인덱스 탐색
Hash Join	12.881 ms	360	큰 동등 조인·정렬 불필요	현재 데이터에서 가장 빠르고 버퍼 접근 최소화
Merge Join	13.837 ms	442	양쪽 입력이 조인 키로 정렬	PK와 order_id 인덱스 순서를 활용

해석

핵심 현재 규모에서는 Hash Join이 가장 효율적이었다. 그러나 NLJ가 항상 나쁜 것은 아니다. 외부 행이 매우 적고 내부 인덱스 선택도가 높으면 NLJ가 유리하며, 이미 정렬된 대량 입력에서는 Merge Join이 안정적이다.

```
SELECT
  o.customer_id,
  count(*) AS item_count,
  sum(oi.line_total) AS revenue
FROM ecom.orders o
JOIN ecom.order_items oi
  ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
GROUP BY o.customer_id;
```

부록 B. Materialized View

반복 요청이 많은 일별 **GMV** 집계를 미리 저장해 조회 비용을 줄이는 구조다. 일반 **View**와 달리 결과를 물리적으로 보관하므로 원본 변경 후 명시적으로 갱신해야 한다.

생성 및 갱신 스크립트

```
CREATE MATERIALIZED VIEW IF NOT EXISTS ecom.mv_daily_gmv AS
SELECT
  date_trunc('day', o.order_ts) AS day,
  sum(oi.line_total) AS gmv
FROM ecom.orders o
JOIN ecom.order_items oi
  ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
GROUP BY date_trunc('day', o.order_ts);
```

```
CREATE INDEX IF NOT EXISTS idx_mv_daily_gmv_day
ON ecom.mv_daily_gmv(day);
```

```
REFRESH MATERIALIZED VIEW ecom.mv_daily_gmv;
```

```
SELECT
  count(*) AS days,
  sum(gmv) AS gmv,
  max(day) AS latest_day
FROM ecom.mv_daily_gmv;
```

MATERIALIZED VIEW 자동 갱신 검증

```
1. BASELINE
gmv
-----
6987702.67
total rows: 1

2. TEST INSERT 후, REFRESH 전 (MV는 STALE)
source_gmv | mv_gmv | test_amount
-----+-----+-----
6987826.12 | 6987702.67 | 123.45
total rows: 1

3. REFRESH 후
source_gmv | mv_gmv
-----+-----
6987826.12 | 6987826.12
total rows: 1

4. ROLLBACK 후 보존 확인
test_orders | test_items | mv_gmv
-----+-----+-----
0           | 0           | 6987702.67
total rows: 1
```

검증 결과 트랜잭션 안에서 테스트 매출 123.45를 추가하자 원본 **GMV**만 증가하고 **MV**는 이전 값을 유지했다. **REFRESH** 후 두 합계가 같아졌으며, 마지막 **ROLLBACK**으로 테스트 행이 남지 않았음을 확인했다.

운영 판단 **Materialized View**는 자동 갱신되지 않는다. 일 배치나 적재 완료 이벤트 뒤에 **REFRESH**를 실행하고, 조회 중단을 줄이려면 **day** **UNIQUE** 인덱스와 **CONCURRENTLY**를 검토한다.

부록 C. 최종 인사이트

랜덤 데이터에서 배운 점

- 숫자 자체보다 정의가 먼저다. 매출 상태·기간·주문 단위를 합의하지 않으면 같은 요청에도 다른 답이 나온다.
- 1:N 조인은 주문 수와 AOV를 쉽게 부풀린다. 주문별 선집계는 성능보다 먼저 정확성을 지키는 장치다.
- 튜닝 후 SQL이 항상 더 빠르지는 않았다. 작은 테이블에서는 Seq Scan이나 CTE 인라인이 합리적이며, 실행계획과 버퍼를 함께 봐야 한다.
- 랜덤 시드는 결과값을 바꾸지만 검증 가능한 로직은 유지된다. 따라서 전달 자료에는 기준시각·조건·단위·가정을 함께 적어야 한다.
- 상관관계와 인과관계를 구분해야 한다. 쿠폰 주문의 높은 AOV는 쿠폰 효과일 수도 있지만 고가 상품 편향이나 시드 생성 규칙의 영향일 수도 있다.